Functions II — Return Values

Lecture 9

CS 1001

Kirk Duran

Today

- Treat a value-returning function call as an expression that itself produces a value.
- Trace argument evaluation and positional parameter binding explicitly.
- Model one function call using a temporary local execution environment.
- Define the semantics of `return`: evaluate a value, end the call, and resume the caller.
- Distinguish returning a value from printing a value.
- Trace repeated calls to the same function without conflating their local bindings.

Key idea

Lecture 8 treated a function as an abstraction. Today we open one call and trace exactly how its value is produced.

Part 1: A Function Call Is an Expression

Expressions Produce Values

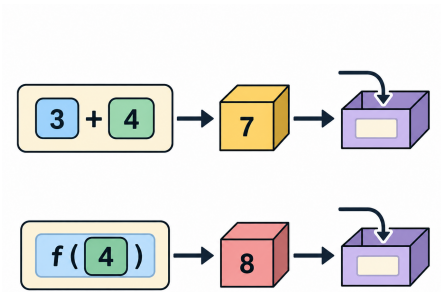
- An expression is evaluated to obtain a value.
- Assignment then binds a name to that resulting value.
- This model does not change when the expression is a function call.

Arithmetic expression

```
answer = 3 + 4
```

Function-call expression

```
answer = double(4)
```



The Returned Value Can Participate in a Larger Expression

Assignment

```
x = double(4)
```

```
double(4) -> 8
x -> 8
```

Arithmetic context

```
y = 3 + double(4)
```

```
3 + 8 -> 11
y -> 11
```

Comparison context

```
double(4) > 5
```

```
8 > 5 -> True
```

Boundary for today

We will use ordinary expressions as arguments. Function calls nested inside other function calls are deferred to Lecture 10.

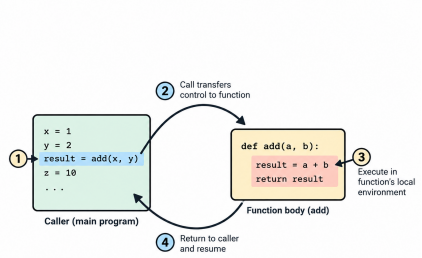
Part 2: The Function-Call Evaluation Model

A Call Temporarily Changes the Flow of Execution

- Initial code normally executes sequentially.
- At a function call, evaluation transfers to the function body.
- The body executes in its own local execution environment.
- When the call returns, the suspended caller resumes.

Control-flow shape

caller $\rightarrow$ function body $\rightarrow$ caller resumes



The Call-Evaluation Procedure

1. Encounter the function-call expression.
2. Evaluate each argument expression to obtain argument values.
3. Create a new local execution environment for this call.
4. Bind each parameter name to its corresponding argument value.
5. Execute statements in the function body sequentially.
6. When `return expression` is encountered, evaluate that expression.
7. End the current call and make the resulting value available to the caller.
8. Resume evaluation at the location of the original call.

Key idea

This procedure is the mental model we will repeatedly test with manual traces and the debugger.

Trace One Call from Start to Finish

Program

```
def double(n:  int) -> int:
   result = n * 2
   return result

x = 4
answer = double(x)
```

Before the call

`x -> 4    answer:  not yet bound`

In class

Predict the values of `n`, `result`, the returned value, and finally `answer`.

The Argument Value Becomes the Parameter Binding

Caller

`x -> 4`

Argument expression

`x` ⟶ `4`

Call to `double`

`n -> 4`

- The name `x` is not renamed to `n`.
- The expression `x` is evaluated.
- Its resulting value is bound to `n`.

Key idea

Data crosses the function boundary as a value; caller variable names do not become parameter names.

The Function Body Then Executes Sequentially

At function entry

`n -> 4`

Execute the first statement

`result = n * 2`

`n * 2` $\longrightarrow$ `4 * 2` $\longrightarrow$ `8`

therefore `result -> 8`

- Parameters already have bindings when the function body begins.
- Local variables appear only when their assignment statements execute.

Returning Reconnects the Function with Its Caller

Inside `double`

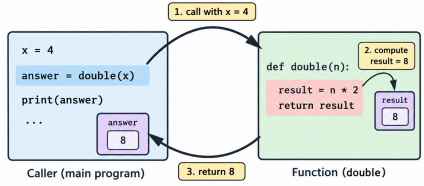
`return result`

`result -> 8`

returned value: `8`

Back in the caller

`answer = double(x)`
conceptually becomes
`answer = 8`



Part 3: Arguments and Parameters

Arguments Are Expressions; Parameters Are Local Names

Definition

An **argument** is an expression supplied at a function-call site.

Definition

A **parameter** is a local name declared in a function definition.

Call site

```
difference(x, y)
```

Definition

```
def difference(first, second):
```

Key idea

Arguments are evaluated; parameters receive bindings to the resulting values.

Multiple Positional Arguments Bind by Position

Function and call

```
def difference(first:  int, second:  int) -> int:
   return first - second

answer = difference(10, 4)
```

Position	Argument value	Parameter binding
1	10	`first -> 10`
2	4	`second -> 4`

Changing Argument Order Changes the Bindings

Call A

```
difference(10, 4)
```

```
first -> 10
second -> 4
```

result: 6

Call B

```
difference(4, 10)
```

```
first -> 4
second -> 10
```

result: -6

Positional semantics

Python does not infer intended roles from the values. For positional arguments, position determines the parameter binding.

Caller Names Do Not Need to Match Parameter Names

Program

```
def rectangle_area(width:  int, height:  int) -> int:
   return width * height

w = 5
h = 3
area = rectangle_area(w, h)
```

Data movement

```
w -> 5 -> width     h -> 3 -> height
```

Key idea

Names are meaningful within their own context. The values, not the caller's variable names, are passed into the call.

Trace an Argument Expression Pedantically

Statement

```
answer = square(x + 2)
```

1. Evaluate `x`: obtain 3.
2. Evaluate `3 + 2`: obtain 5.
3. Bind `n -> 5` for this call.
4. Evaluate `n * n`: obtain 25.
5. Return 25 to the caller.
6. Bind `answer -> 25`.

Part 4: The Semantics of `return`

`return` Ends the Current Function Call

- Once an executed `return` is reached, the current call is complete.
- Control transfers back to the caller.
- The caller receives the returned value as the value of the call expression.

Conceptual rule

`return expression` $\Longrightarrow$ evaluate expression $\rightarrow$ end call $\rightarrow$ resume caller

Terminology

Avoid saying that `return` "prints the answer" or "stores the answer." Neither description captures its semantics.

The Caller Determines What Happens to the Returned Value

Store it

```
x = square(4)
```

```
x -> 16
```

Use it

```
y = 2 * square(4)
```

```
y -> 32
```

Ignore it

```
square(4)
```

value produced, but not bound

Key idea

A function returns a value to its caller; the surrounding caller expression determines how that value is used.

Printing and Returning Are Different Operations

Return a value

```
def square_return(n):
  return n * n
```

- Produces a value for the caller.
- The caller can store or combine that value.

Print a value

```
def square_print(n):
  print(n * n)
```

- Produces visible output as a side effect.
- Does not explicitly return the printed number.

A Function Without Explicit `return` Returns `None`

Example

```
def square_print(n):
    print(n * n)

result = square_print(4)
```

Visible output

16

Returned value

`None`

Key idea

Visible output and a returned value are independent concepts.

Prediction: Return or Print?

Program

```
def show_double(n):
    print(n * 2)

def get_double(n):
    return n * 2

a = show_double(6)
b = get_double(6)
```

In class

Predict (1) what is displayed, (2) the final binding of `a`, and (3) the final binding of `b`. Explain each using call semantics rather than intuition.

Part 5: Local Bindings and Independent Calls

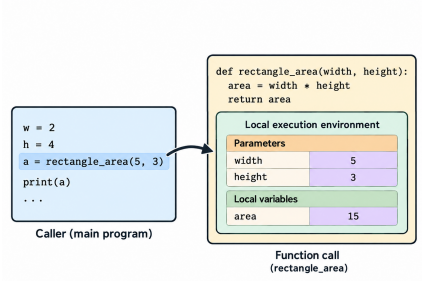
Each Call Has a Local Execution Environment

Definition

A **local execution environment** is the set of parameter and local-variable bindings associated with one active function call.

During `rectangle_area(5, 3)`

```
width -> 5
height -> 3
area -> 15
```



Local Variables Appear as Their Assignments Execute

Function

```
def rectangle_area(width, height):
   area = width * height
   return area
```

Execution point	Local bindings
Function entry	`width -> 5, height -> 3`
After assignment	`width -> 5, height -> 3, area -> 15`
After return	call no longer active

Function-Local Bindings Have Call-Limited Lifetime

- Parameter bindings are created for a specific call.
- Local-variable bindings are created as statements in that call execute.
- When the call completes, that local execution environment is no longer active.
- The returned value may survive because the caller can bind or use it.

Before, during, after

caller bindings $\rightarrow$ caller + local call bindings $\rightarrow$ caller bindings + returned result

Scope preview

Lecture 10 will examine what happens when more than one function call is active at the same time.

Calling the Same Function Twice Creates Two Separate Calls

Program

```
def increase(value:  int) -> int:
   result = value + 1
   return result

first = increase(4)
second = increase(10)
```

First call

```
value -> 4
result -> 5
```
returns 5

Second call

```
value -> 10
result -> 11
```
returns 11

Trace Tables Need a Location Column Once Functions Appear

Step	Location	Statement	Relevant bindings
1	caller	`x = 4`	`x -> 4`
2	`double`	function entry	`n -> 4`
3	`double`	`result = n * 2`	`result -> 8`
4	`double`	`return result`	returns 8
5	caller	assignment completes	`answer -> 8`

Key idea

A useful trace records both *what executes* and *where it executes*.

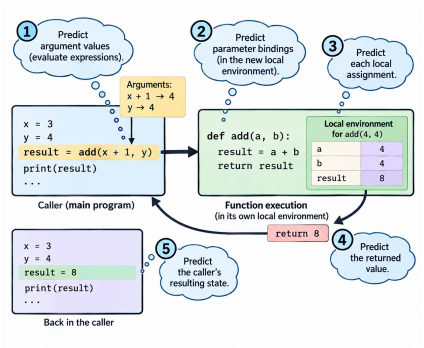
Part 6: Manual Tracing and Debugger Verification

Manual Prediction Comes Before the Debugger

1. Predict argument values.
2. Predict parameter bindings.
3. Predict each local assignment.
4. Predict the returned value.
5. Predict the caller's resulting state.

Key idea

The debugger is evidence used to test a mental model, not a replacement for reasoning.



`Step Into` Exposes Function-Call Execution

- **Step Over** executes a function call without tracing through its body line by line.
- **Step Into** transfers debugger focus into the called function.
- Inside the function, inspect parameter bindings and local variables as they appear.
- When the function returns, observe execution resume at the caller.

Lab question

Does the debugger's Variables frame match the bindings predicted by your manual trace?

Worked Trace: Discounted Price

Program

```
def discounted_price(price:  float, rate:  float) -> float:
    discount = price * rate
    final = price - discount
    return final

original = 80.0
sale = discounted_price(original, 0.25)
```

In class

Trace the complete call. Record the argument values, parameter bindings, `discount`, `final`, returned value, and final binding of `sale`.

Worked Trace: Reason from Values, Not Similar Names

Program

```
def difference(first:  int, second:  int) -> int:
   result = first - second
   return result

first = 20
second = 7
answer = difference(second, first)
```

In class

Determine the parameter bindings without relying on the repeated names. Begin by evaluating the two argument expressions in their written order.

Expected discipline

Caller names and parameter names may be identical, different, or misleading. Positional evaluation still determines the bindings.

Common Failure Modes in Function Tracing

Incorrect model	Correction
"The argument variable becomes the parameter."	The argument expression is evaluated; its value is bound to the parameter.
"`return` prints the result."	`return` produces the value of the call for the caller.
"A local variable exists as soon as the function starts."	A local binding appears when its assignment executes.
"The next call remembers the previous locals."	Each call creates a distinct local execution environment.
"Assignment happens before the call finishes."	The right-hand-side call must finish before assignment can bind its result.

The Boundary of Today's Model

Today: at most one active user-defined function call

caller $\longrightarrow$ function $\longrightarrow$ caller resumes

- We can trace calls with multiple parameters and local variables.
- We can evaluate ordinary expressions before and after a call.
- We can call the same function repeatedly, one call after another.

Next conceptual problem

What if a function calls another function *before the first call has finished*?

Takeaways

- A value-returning function call is an expression and therefore evaluates to a value.
- Argument expressions are evaluated before their values are bound to parameters.
- Positional arguments correspond to parameters by position, not by caller variable name.
- One call has a temporary local execution environment containing parameter and local-variable bindings.
- `return` evaluates an expression, ends the current call, and supplies that value to the caller.
- Printing a value is not equivalent to returning it; functions without explicit `return` return `None`.
- Separate calls to the same function create separate local bindings.

Key idea

Mental model: **evaluate arguments** → **bind parameters** → **execute locally** → **return a value** → **resume the caller**.